Wavelets and Autoencoders are All You Need

Anonymous Author(s)
Affiliation
Address
email

Abstract

N-BEATS (Oreshkin et al., 2020) broke new ground in univariate time series forecasting by using fixed basis functions in a neural basis expansion in which the network learns coefficients for a frozen basis function and then expands those coefficients into the desired forecast and backcast lengths through a linear transformation. Their innovation came at an extreme cost, as a typical N-BEATS model can range from 25M–44M parameters. In this paper we extend the idea of solving for coefficients of a frozen basis function but, instead of relying on a global Fourier function, we use wavelets, which provide localization in that they can identify and locate components of the global signal — a key requirement in time series analysis algorithms. Wavelets have been used extensively in signal analysis for feature extraction, denoising, and multi-resolution analysis. Likewise, autoencoders are useful constructs for identifying key characteristics of a signal while filtering out non-essential transient components, and are useful in domains such as denoising signals and stock trading. We introduce a variety of novel block types that use a combination of discrete wavelet transforms (DWT) and autoencoders. Our novel architecture improvements reduce parameter counts by 60–95% over N-BEATS while surpassing or equalling prior published results without ensembling. In addition, we introduce a family of combined TrendWavelet block types that combine a low-pass trend basis with a high-pass wavelet basis to produce <1M parameter models that score within 0.5% sMAPE of 19M–43M parameter baseline N-BEATS models, representing a 12–80× compression with little to no loss of accuracy. Furthermore, we show that these new block constructs are transferable to other architectures like NHiTS and Transformers with equally impressive reductions in parameter counts and competitive accuracies.

1 Introduction and Prior Work

1.1 Introduction

Time series forecasting and signal analysis are two names for the same thing, even though practitioners in each discipline use their own terminologies: moving average vs. finite impulse response (FIR) filter, autoregressive filter vs. infinite impulse response (IIR) filter, seasonal decomposition vs. band-pass filter of seasonal frequencies. A time series dataset (sales figures, temperature, stock prices) is just a discrete set of sampled data. Likewise, all deep learning problems can be reconsidered as signal analysis problems. For example, image recognition in deep learning is often performed by sampling an image and passing the samples through a convolutional filter, similar in principle to what one might do in Photoshop when applying a sharpen filter. Deep learning, then, can be seen as a type of probabilistic signal analysis. Why, then, don't we use more techniques developed in signal analysis in machine learning?

Signal analysis tools like the Discrete Fourier Transform and the Discrete Wavelet Transform were developed as techniques used in signal processing, migrated into forecasting, and are trickling into

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

machine learning. N-BEATS was a pioneer of sorts in bringing some traditional signal analysis tools into AI with its use of basis expansion blocks. The N-BEATS Seasonality and Trend blocks are literally signal-decomposition operators applied inside a neural network. The novelty was using the neural network to learn which decomposition coefficients best reconstruct the observed signal.

1.2 Wavelets

Fourier analysis decomposes a signal into a sum of sine and cosine waves of fixed frequency. It is a useful tool for analyzing stationary signals whose frequency content does not change over time. Real-world signals routinely break that mold, however. For example, a stock price may trend smoothly following the normal sales cycle and then spike abruptly because of a news article, a comment made by a prominent person in the news, or when a large hedge fund liquidates a considerable position. Fourier bases represent these transient features poorly and tend to smear localized events across the other frequency components of the signal.

Wavelets address this problem by providing basis functions that are localized in both time and frequency. Where a sine wave oscillates forever, a wavelet is a brief oscillatory burst with exponentially decaying tails and a zero mean. They oscillate in a particular pattern defined by their mother wavelet and then decay to zero.

The mother wavelet $\psi(t)$ is scaled and translated to produce a family of basis functions:

$$\psi_{j,k}(t) = 2^{j/2}\,\psi(2^j t - k)$$

where j controls scale (frequency resolution) and k controls translation (time position). Short, high-frequency wavelets have narrow time resolution; long, low-frequency wavelets have narrow frequency resolution. This makes wavelets substantially more effective for real-world time series analysis, where variance and frequency content change over time.

A discrete wavelet transform (DWT) brings the continuous wavelet into the digital domain. DWTs compute the inner product of a signal with a basis family, producing a set of orthonormal coefficients that can then be used to reconstruct the original signal.

Wavelets have been used sparsely in machine learning. WaveNet (2022, arXiv:2505.11781) learns a filter-bank specialized to fit non-stationary signals by learning the basis function through decomposing its n-th order derivatives and then learning an affine transformation that lets the network suppress and cross-mix frequencies in training.

MultiWave (arXiv:2306.10164) is a model-agnostic preprocessor around any neural network architecture. It takes the concept of machine-learning/signal-processing equivalency to heart by preprocessing input data with a DWT, grouping the outputs by frequency, and then feeding the groups into any neural network (Transformer, FFN, CNN, etc.), feeding different groups into different networks. The networks' outputs are combined and an inverse DWT is applied to obtain the prediction.

W-Transformers (arXiv:2209.03945v1) uses a similar approach by preprocessing time-series data with a discrete overlapping wavelet transform and then feeding the results into a collection of transformer models to analyze the decomposed signals independently.

1.3 Autoencoders

An autoencoder is a neural network trained to reproduce its input. It has an encoder that maps an input x into a smaller-dimensional latent code z, and a decoder that maps z back into a reconstructed version of the original input $\hat{x}$:

```
x -> encoder -> z -> decoder -> x_hat
loss = reconstruction_error(x, x_hat) + optional_regularization
```

The premise is that the model is constrained by the reduced dimensionality of the latent space. Therefore, since the decoder network does not have access to the original signal, a perfect copy is difficult. Thus the network is pushed to learn how to reconstruct the original signal from the information it can extract from the latent representation. The classic paper of Hinton and Salakhutdinov (Hinton & Salakhutdinov, 2006) showed that autoencoders generalize PCA to non-linear domains. Autoencoders are also used widely for self-supervised pretraining, generative modeling, and image denoising and generation.

2 Model Description

N-BEATS (Oreshkin et al., 2020) analyzes a time series through a branching stack of parallel blocks, each contributing a backcast $\hat{\mathbf{x}}_\ell$ and forecast $\hat{\mathbf{y}}_\ell$ via a learned linear mapping from a shared 4-layer fully connected backbone:

$$\mathbf{h}_\ell = f_{\text{bb}}(\mathbf{x}_\ell^{\text{in}}), \quad \hat{\mathbf{x}}_\ell = g_b(\boldsymbol{\theta}^b), \quad \hat{\mathbf{y}}_\ell = g_f(\boldsymbol{\theta}^f)$$

where $\mathbf{x}_\ell^{\text{in}} = \mathbf{x} - \sum_{k<\ell} \hat{\mathbf{x}}_k$ is the residual backcast input to block ℓ, and g_b, g_f are basis expansion generators (defined explicitly in Section 2.3).

All N-BEATS blocks accept a residual backcast (or input if first block) $\mathbf{x} \in \mathbb{R}^L$ and return $(\hat{\mathbf{x}}, \hat{\mathbf{y}}) \in \mathbb{R}^L \times \mathbb{R}^H$.

2.1 Backbone Blocks

2.1.1 RootBlock

The legacy RootBlock is a 4-layer, uniform-width FFN with ReLU activation σ:

$$\mathbf{h} = \sigma(\mathbf{W}_4\, \sigma(\mathbf{W}_3\, \sigma(\mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) + \mathbf{b}_4)\,, \quad \mathbf{W}_i \in \mathbb{R}^{U \times U}$$

No bottleneck is imposed; all layers share trunk width U.

2.1.2 AERootBlock

The AERootBlock is a drop-in replacement for the legacy RootBlock backbone. It introduces an hourglass-shaped autoencoder that routes all information through a low-dimensional latent bottleneck $z \in \mathbb{R}^d$:

$$\mathbf{h} = f_{\text{dec}}(f_{\text{enc}}(\mathbf{x})), \quad f_{\text{enc}} : \mathbb{R}^L \to \mathbb{R}^{U/2} \to \mathbb{R}^d, \quad f_{\text{dec}} : \mathbb{R}^d \to \mathbb{R}^{U/2} \to \mathbb{R}^U$$

The bottleneck acts as a signal denoiser, regularizer, and feature extractor. The decoder must reconstruct both output paths from z, forcing the backbone to discard irrelevant information and retain the most informative features.

The parameter count of the backbone reduces to $O(UL/2 + Ud/2 + Ud/2 + U^2/2)$. At $U = 512$, $L = 480$, $d = 16$, this is approximately 147K — a 5.5× reduction relative to RootBlock at equivalent U. The interface to any downstream projection head remains identical: both backbones produce $h \in \mathbb{R}^U$.

2.1.3 AERootBlockLG

The LG variant RootBlock introduces a learnable sigmoid vector and scales the interior latent representation element-wise:

$$z_{\text{gated}} = \sigma(\gamma) \odot z$$

Because $\sigma(\gamma_i)$ at its limits approaches zero, the network can shut off unnecessary latent dimensions, effectively performing a soft band-pass filter. This is particularly useful when the optimal bottleneck size is unknown: one can select a lower latent dimension because the network only uses the dimensions it needs. In testing, this manifests as networks using AERootBlocks needing a latent dimension $d = 32$ for the M4 dataset, but AELG-based networks needing only $d = 16$. For example, a 10-stack TrendWavelet block with AELG root and a latent dimension of 32 using a Daubechies-3 wavelet (`TWAELG_10s_ld32_db3`) uses only 0.48M total parameters while remaining competitive with 15M+ RootBlock models.

2.1.4 Parameter efficiency

The standard width parameters (`g_width`=512, `s_width`=2048, `t_width`=256, `ae_width`=512) govern the main trunk width U for each block family. The `latent_dim` d is an additional hyperparameter that controls only the AE bottleneck and does not change the output dimension $\mathbf{h} \in \mathbb{R}^U$ seen by downstream projection heads.

The backbone parameter count formula for each family is:

Table 1: Backbone architecture comparison.

Backbone	Layer topology	Latent operation	Extra params
RootBlock	input $\to U \to U \to U \to U$	None	Baseline
AERootBlock	input $\to U/2 \to d \to U/2 \to U$	Deterministic pass-through	Negligible
AERootBlockLG	Same as AE	$z \cdot \sigma(\gamma)$	d (gate vector)

Table 2: Backbone parameter counts at default widths ($L = 480$).

Backbone	Width U	Latent d	Backbone params	vs. RootBlock
RootBlock (Generic)	512	—	∼810K	1.0×
AERootBlock (Generic)	512	32	∼163K	5.0× fewer
AERootBlock (Generic)	512	16	∼147K	5.5× fewer
AERootBlockLG (Generic)	512	16	∼147K + 16	≈5.5× fewer
RootBlock (Trend)	256	—	∼319K	1.0× (Trend)
AERootBlock (Trend)	256	16	∼46K	6.9× fewer
AERootBlockLG (Trend)	256	16	∼46K + 16	≈6.9× fewer

- **RootBlock**: $UL + 3U^2$ — four uniform-width layers, no bottleneck.
- **AERootBlock**: $UL/2 + Ud + U^2/2$ — parameters scale roughly as $O(U \cdot d)$ at the bottleneck, which is minor compared to the main width cost.
- **AERootBlockLG**: same as AERootBlock plus d gate parameters — negligible overhead; practical benefit is that d can be halved relative to AERootBlock without accuracy loss.

Because `latent_dim` is typically 5–32, AE variants achieve comparable forecast accuracy with dramatically fewer total backbone parameters than their RootBlock counterparts when `latent_dim` is small and width is constrained. Table 2 gives concrete counts at default widths and a representative backcast length of $L = 480$.

The compression ratio grows with width: wider trunks benefit more from the hourglass bottleneck because the three large $U \times U$ layers in RootBlock scale quadratically while the AE bottleneck scales linearly in d. At Generic/Wavelet width ($U = 512$, $d = 16$) the ratio reaches 5.5×; at Trend width ($U = 256$, $d = 16$) it reaches 6.9×. This makes AE backbones attractive for TrendWavelet and TrendWaveletGeneric configurations, where the Trend trunk width is narrow by design and every saved parameter can be reinvested in stack depth or wavelet family selection, which improves expressivity of the model.

2.2 Branch-specific encoder-decoder designs

The backbone variants described above produce a single shared hidden representation $\mathbf{h}$ that is then split by separate linear heads into backcast and forecast coefficient vectors. A complementary design gives each output branch its own dedicated encoder-decoder sub-network. The `AutoEncoderAE` and `GenericAEBackcastAE` blocks follow this pattern: after the shared AE trunk compresses the input into a trunk latent code z, two independent hourglass sub-branches — one for the backcast path and one for the forecast path — each re-encode z through their own bottleneck before projecting to the output length.

$$z \xrightarrow{\text{enc}_b} z_b \xrightarrow{\text{dec}_b} \hat{\mathbf{x}}, \qquad z \xrightarrow{\text{enc}_f} z_f \xrightarrow{\text{dec}_f} \hat{\mathbf{y}}$$

This gives the network two independent compression points per block. The trunk bottleneck enforces a shared representation of the input window; the branch bottlenecks then clean up each signal independently. The two objectives are separated downstream again to impart more diversity into the learned parameters.

The `latent_dim` hyperparameter controls the branch-local bottleneck size in these blocks, while `thetas_dim` remains reserved for explicit coefficient projections such as the `GenericAEBackcast*` forecast head. `AutoEncoderAE` omits `thetas_dim` entirely and lets both branch decoders project directly to the target lengths.

Table 3: Single-trunk vs. branch-specific AE designs.

Block	Shared trunk	Backcast branch	Forecast branch	`thetas_dim` role
`GenericAE`	AERootBlock	shared $\mathbf{h} \to$ linear	shared $\mathbf{h} \to$ linear	unused
`AutoEncoderAE`	AERootBlock	$z \to$ enc-dec $\to$ output	$z \to$ enc-dec $\to$ output	unused
`GenericAEBackcastAE`	AERootBlock	$z \to$ enc-dec $\to$ output	$z \to$ enc-dec $\to \boldsymbol{\theta}^f \to$ basis	coefficient projection

The branch-specific design adds approximately $4 \cdot U_b \cdot d_b$ parameters per branch, where U_b is the branch width and d_b the branch latent size. For typical settings ($U_b = U/2$, $d_b = d$), this doubles the AE parameter budget relative to a shared-head block, but keeps the total well below the RootBlock baseline since both the trunk and the branches remain in the hourglass regime.

2.3 Wavelet Family Blocks

The legacy N-BEATS basis-expansion blocks predict coefficient vectors and expand them through a frozen basis into backcast and forecast outputs matching the target lengths. The original interpretable version uses polynomial trend and Fourier seasonality bases. The wavelet blocks replace the Fourier seasonality basis with a fixed, orthonormal discrete wavelet basis. Each block first maps the input window through a RootBlock variant, producing a hidden representation $\mathbf{h}$. Separate trainable linear heads then produce backcast and forecast coefficient vectors,

$$\boldsymbol{\theta}^b = \mathbf{W}_b \mathbf{h}, \qquad \boldsymbol{\theta}^f = \mathbf{W}_f \mathbf{h}$$

For each target length $T \in \{L, H\}$, a wavelet synthesis matrix is constructed at model initialization by applying inverse DWT reconstruction to unit impulses in each wavelet coefficient band. This raw synthesis matrix is then orthogonalized via SVD, and the rows of $V^\top$ form the fixed orthonormal bases $\boldsymbol{\Psi}_b \in \mathbb{R}^{k \times L}$ and $\boldsymbol{\Psi}_f \in \mathbb{R}^{k \times H}$ (one per path). The basis expansion generators therefore evaluate to

$$g_b(\boldsymbol{\theta}^b) = \boldsymbol{\theta}^b \, \boldsymbol{\Psi}_b = \hat{\mathbf{x}}, \qquad g_f(\boldsymbol{\theta}^f) = \boldsymbol{\theta}^f \, \boldsymbol{\Psi}_f = \hat{\mathbf{y}}$$

Because $\boldsymbol{\Psi}_b$ and $\boldsymbol{\Psi}_f$ are frozen at initialization and stored as non-trainable buffers, inference requires only a single matrix multiply per path, which GPUs excel at. A full DWT is never computed at training or inference time. Implementation details are given in Appendix B.

2.3.1 Composition blocks: TrendWavelet and TrendWaveletGeneric

The strongest single-block family in our benchmarks combines a frozen polynomial trend basis with a frozen orthonormal wavelet basis in a single additive decomposition. The `TrendWavelet` block produces the coefficient vector $\boldsymbol{\theta} \in \mathbb{R}^{p+k}$ from the shared hidden representation $\mathbf{h}$, then partitions and expands it:

$$\hat{\mathbf{y}} = \underbrace{\boldsymbol{\theta}^f_{1:p} \, \mathbf{T}_f}_{\text{trend}} + \underbrace{\boldsymbol{\theta}^f_{p+1:p+k} \, \boldsymbol{\Psi}_f}_{\text{wavelet}}$$

where $\mathbf{T}_f \in \mathbb{R}^{p \times H}$, $\mathbf{T}_b \in \mathbb{R}^{p \times L}$ is the Vandermonde polynomial basis of order p (`trend_thetas_dim`), $\boldsymbol{\Psi}_f \in \mathbb{R}^{k \times H}$, $\boldsymbol{\Psi}_b \in \mathbb{R}^{k \times L}$ is the orthonormal wavelet basis of rank k (`basis_dim`), and the same additive structure holds for the backcast path. The two bases are complementary by construction: the polynomial captures smooth low-frequency trend while the wavelet captures localized oscillatory residuals.

`TrendWaveletGeneric` extends this to a three-way decomposition by appending a third learned (data-driven) branch of rank r (`generic_dim`):

$$\hat{\mathbf{y}} = \boldsymbol{\theta}^f_{1:p} \, \mathbf{T}_f + \boldsymbol{\theta}^f_{p+1:p+k} \, \boldsymbol{\Psi}_f + \boldsymbol{\theta}^f_{p+k+1:p+k+r} \, \mathbf{G}_f$$

where $\mathbf{G}_f \in \mathbb{R}^{r \times H}$ is a trainable rank-r basis matrix. The two structured bases remain frozen; only coefficients and the generic basis are learned. This gives the block a fall-back data-driven capacity for signal components that neither polynomial nor wavelet bases can represent compactly.

Both block families are fully composable with both AE backbone variants (AERootBlock, AERootBlockLG), yielding the `TrendWaveletAE`, `TrendWaveletAELG`, `TrendWaveletGenericAE`, and `TrendWaveletGenericAELG` configurations benchmarked in Section 4.

Table 4: Datasets and evaluation parameters.

Dataset	Frequency	Series	Horizon H	Lookback L	Primary metr
M4 (Y/Q/M/W/D/H)	mixed	100,000	$\{6, 8, 24, 13, 14, 48\}$	$L = 5H$; $L_h = mH$, $m \in \{1.5, 10\}$	sMAPE, OW
Tourism (Y/Q/M)	mixed	1,311	$\{4, 8, 24\}$	$L = 5H$	sMAPE
Traffic-96	hourly	862	96	$5H$	MSE, MAE
Weather-96	10-min	21 ch.	96	$5H$	MSE, MAE
Milk	monthly	1	6	$5H$	sMAPE

2.3.2 Asymmetric and tiered wavelet bases

Two additional hyperparameters control how the wavelet basis is oriented relative to the backcast and forecast windows.

Asymmetric wavelet families. When backcast and forecast lengths differ substantially — as in Traffic-96 ($L = 480$, $H = 96$, ratio $5\times$) — the optimal wavelet support length differs between paths. The `backcast_wavelet_type` and `forecast_wavelet_type` parameters allow independent wavelet family selection per path. A long-support wavelet (e.g. Symlet-20) captures broad low-frequency structure in the long backcast; a short-support wavelet (e.g. Daubechies-3, Coiflet-2) provides compact localized bases for the short forecast. When these overrides are omitted, both paths share the same `wavelet_type`.

Tiered basis offset. The `basis_offset` parameter shifts the starting row of the basis matrix selected from $V^\top$, cycling through the SVD spectrum in successive stacks. In a depth-S model, stack ℓ uses rows $[\ell \cdot \Delta, \ell \cdot \Delta + k)$ of the orthonormal basis, where Δ is a stride derived from `basis_offset`. This spreads each stack's basis coverage across different frequency bands, encouraging the doubly residual stack to decompose the signal rather than repeatedly fitting the same spectral region. Tiered offsets are most effective on the Daily M4 period, where they appear in 8 of the top-10 configurations (Section **??**).

3 Experimental Setup

This section describes the datasets, training protocols, architecture sweep, and ablation studies used to evaluate the proposed blocks. Methodology is presented here; results follow in Section 4.

3.1 Datasets

The M4 Competition (Makridakis et al., 2020) dataset is the primary evaluation dataset: six frequencies (Yearly, Quarterly, Monthly, Weekly, Daily, Hourly) over 100,000 series with horizons $H \in \{6, 8, 24, 13, 14, 48\}$. This benchmark requires a single architecture to generalize across regimes that differ in sample size, sampling rate, and seasonality without time-series-specific feature engineering or input scaling. We complement M4 with four out-of-distribution datasets to verify that conclusions transfer beyond M4. These are Tourism (1,311 series, Yearly/Quarterly/Monthly), PeMS Traffic (862 hourly sensors), Weather (21 meteorological channels at 10-min resolution), and the univariate Milk production series.

The paper-sample protocol additionally enforces the N-BEATS in-sample constraint L_h (Oreshkin et al., 2020, Appendix D), restricting valid training windows to the trailing $L_h \cdot H$ time steps of each series; we use $L_h = 1.5$ for Yearly/Quarterly and $L_h = 10$ for the remaining M4 frequencies.

3.2 Training Protocols

Absolute sMAPE differences in basis-expansion benchmarks are often dominated by sampling and learning-rate artifacts rather than by architectural quality. To control for this we evaluate every architecture under **three complementary protocols**. The protocols share the same backbone, loss, and seed budget, but they differ in how training data is presented to the optimizer and in how the learning rate is decayed. A finding is treated as robust when it survives all three.

Sliding-Window Protocol (`comprehensive_sweep_m4.yaml`, the `lightningnbeats` default). Epoch-based budget with batches enumerated by sliding the lookback window over each series. Adam at 10^{-3}, sMAPE loss, max 75 epochs (min 10), early-stopping patience 20 with $\Delta_{\min} = 10^{-3}$, cosine annealing with 15-epoch linear warmup and $\eta_{\min} = 10^{-6}$.

N-BEATS Paper-Sampling Protocol, faithful replication (`comprehensive_m4_paper_sample.yaml`). A direct copy of the original N-BEATS training procedure (Oreshkin et al., 2020): iteration-based budget of 10^5 gradient steps at batch size 1024, with each step drawing (series, anchor) pairs uniformly at random with replacement from the windows that satisfy the L_h constraint. Adam at 10^{-3}, sMAPE loss, MultiStepLR with 10 evenly-spaced milestones and $\gamma = 0.5$ (LR halved every 10% of training). This protocol exists to put our proposed blocks on the same training footing as the published baseline numbers.

N-BEATS Paper-Sampling Protocol, plateau LR variant (`comprehensive_m4_paper_sample_plateau.yaml`). The sampling, batch size, step budget, and early-stopping configuration are identical to the faithful replication. The MultiStepLR is replaced with a more aggressive `ReduceLROnPlateau` scheduler (factor $= 0.5$, patience 3 val-check units, cooldown $= 1$, min_lr $= 10^{-6}$, monitored on `val_loss` every 100 steps). The plateau variant is included because validation-driven LR decay can resolve to a finer effective schedule than the fixed-milestone schedule. In our experiments it changes which architectures look competitive on the Quarterly, Monthly, and Daily M4 cells, while the rankings on Yearly and Weekly are essentially unchanged. By reporting both variants we can separate wins that come from structured priors from wins that come from the LR schedule happening to suit a particular architecture.

Both paper-sample variants share the same sub-epoch validation configuration: `val_check_interval=100`, patience 15 in val-check units (so a minimum of 1,500 steps must elapse before stopping can fire), and $\Delta_{\min} = 10^{-3}$. Without sub-epoch validation, early stopping under iteration-based training fires on warmup-corrupted checks and collapses to `best_epoch` $\in \{0, 1\}$. This is a small methodological correction that we found is necessary for reproducible paper-sample evaluation.

Seeds and statistics. Every (configuration, period, protocol) cell is evaluated over 10 random seeds. Pairwise comparisons use the Wilcoxon signed-rank test on matched seeds with Bonferroni correction; the full multiple-comparison procedure and the verbatim YAML for each protocol are given in Appendix A.

3.3 Architecture Sweep

The central hypothesis under evaluation is:

> *Structured priors (orthonormal wavelet bases and autoencoder bottlenecks) enable 60–95% parameter compression relative to paper-faithful Generic stacks while matching or exceeding their forecasting accuracy.*

To test this, we define a single 112-configuration **Comprehensive Sweep** evaluated under the sliding-window protocol. A curated 53-configuration subset is re-run under both paper-sample variants (faithful MultiStepLR and plateau LR). The families dropped from this subset (BottleneckGeneric, pure VAE) are excluded *a priori* on reproducibility grounds. They exhibit run-to-run divergence under iteration-based sampling that adds variance without informing the comparison axis. The full configuration list is given in Appendix C; the five groups are:

1. **Paper baselines (8 configs).** `NBEATS-G` (homogeneous Generic) and `NBEATS-I+G` (Trend + Seasonality + Generic), each at depths $\{10, 30\}$ and `active_g` $\in \{\text{False}, \text{forecast}\}$. Reproduces Oreshkin et al. (2020) and supplies the upper-bound parameter count.
2. **Homogeneous TrendWavelet stacks (16 configs).** `TrendWavelet` (RootBlock backbone) at depths $\{10, 30\}$, sweeping `wavelet_type` $\in \{\text{haar}, \text{db3}, \text{coif2}, \text{sym10}\}$, `trend_thetas_dim` $\in \{3, 5\}$, and `basis_dim` $\in \{\text{eq_fcast}, 2 \times \text{eq_fcast}\}$. Tests whether one composite basis (polynomial trend + orthonormal DWT) suffices without an explicit Seasonality stack.

3. **Alternating dual-stack architectures (∼30 configs).** A *top-quality* pattern alternating `TrendAELG` and `WaveletV3AELG`, and a *stability* pattern alternating `TrendAE` and `WaveletV3AE`. Each is swept across the four wavelets at depths $\{10, 30\}$. The two halves act as separate doubly-residual specialists rather than a single fused block.

4. **Efficiency bottlenecks (∼12 configs).** `TrendWaveletAE` and `TrendWaveletAELG` at depth 10, with `latent_dim` $\in \{8, 16, 32\}$. The direct test of the compression hypothesis: does an AE bottleneck of dimension $d \ll$ `units`$/2$ preserve accuracy at a fraction of the dense-MLP parameter count?

5. **Generic-AE controls and weight-sharing variants (∼46 configs).** `GenericAE`, `GenericAELG`, and `TrendWaveletGenericAELG` at varied depths and latent dimensions, plus eight `share_weights=False` variants. Isolates AE compression *without* wavelet structure (groups 2–4 confound the two) and quantifies the cost of removing weight sharing.

3.4 Ablation: `active_g`

The `active_g` parameter is a `lightningnbeats` extension that gates whether the final linear projection of Generic-family blocks passes through the block's nonlinearity (`active_g='forecast'`) or returns a raw linear map (`active_g=False`, the paper-faithful setting). We evaluate `active_g` $\in \{\text{False}, \text{forecast}\}$ across the GenericAELG group at depths $\{10, 30\}$, holding all other hyperparameters fixed and sweeping under both protocols. The aim is to isolate the impact of architectural regularization on Generic stack stability, independent of the wavelet and AE structure introduced elsewhere. The companion `skip_distance` ablation (cross-stack residual injection cadence) and the `learned_gate` magnitude analysis on AELG blocks are deferred to Appendix D.

3.5 NHiTS Transferability

To confirm that the proposed blocks are not specific to the N-BEATS doubly-residual stack, we re-run a curated subset of the novel families on the NHiTS backbone (Challu et al., 2023), which adds multi-rate input pooling and hierarchical forecast interpolation around the same block interface. The benchmark (`run_nhits_benchmark.py`) covers Weather-96 and Traffic at horizons $\{96, 192, 336, 720\}$ with MSE loss, batch size 256, max 100 epochs, patience 10, $L = 5H$, and 8 seeds. The block sweep includes `Generic`, `GenericAELG`, `BottleneckGenericAELG`, alternating `TrendAELG + Sym2OV3AELG`, and `TrendWaveletAELG`, paired with their NHiTS-pooled counterparts using horizon-adaptive pooling/interpolation schedules. When block-level effects survive the backbone change, this is evidence that the structured priors are driving them rather than interactions with N-BEATS-specific residual scheduling.

4 Results

Per-period leaderboards, parameter-efficiency frontiers, and ablations are reported here. Detailed tables are deferred to Appendix E.

5 Limitations

The architectural sweep is centered on the M4 benchmark; while we include Tourism, Traffic, Weather, and Milk as out-of-distribution checks, the breadth of structured-prior conclusions outside M4 is bounded by the size of those auxiliary sweeps. The compute budget supported 10 seeds per cell on a single-author setup; some 1–2 sigma effects therefore retain residual seed sensitivity. Pure VAE-family blocks were observed to diverge under iteration-based paper sampling and are excluded *a priori*; we do not claim VAE backbones are unworkable in general, only that they were not stabilized within this study. Finally, all reported gains assume a single architecture is trained per dataset/period; ensembling, common in published M4 results, is intentionally outside scope to keep the parameter-count comparison clean.

References

References

Challu, C., Olivares, K.G., Oreshkin, B.N., Ramirez, F.G., Canseco, M.M., & Dubrawski, A. (2023) NHiTS: Neural Hierarchical Interpolation for Time Series Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(6):6989–6997.

Hinton, G.E. & Salakhutdinov, R.R. (2006) Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020) The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74.

Oreshkin, B.N., Carpov, D., Chapados, N., & Bengio, Y. (2020) N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting. *International Conference on Learning Representations (ICLR)*.

A Paper-Sample Protocol Details

This appendix documents the two paper-sample training protocols in full so that the M4 numbers in Section 4 can be reproduced from a single YAML file. Both variants share the sampling strategy and step budget of the original N-BEATS training procedure; they differ only in the learning-rate scheduler.

A.1 Faithful replication (MultiStepLR)

The faithful variant (`comprehensive_m4_paper_sample.yaml`) is intended as a direct copy of the training procedure described in Oreshkin et al. (2020, Section 5.2 and Appendix D), with no deviations except the small validation-cadence correction described below.

Sampling. Each gradient step draws a batch of 1024 (series, anchor) pairs uniformly at random *with replacement* from the training pool. A series is eligible at step t if it has at least $L + H$ observations; for a chosen series of length T, the anchor index is sampled uniformly from $[T - L_h \cdot H,\ T - H]$, restricting valid windows to the trailing $L_h \cdot H$ time steps. Per-period L_h is fixed to the values in Table 4: $L_h = 1.5$ for Yearly and Quarterly, $L_h = 10$ for Monthly, Weekly, Daily, and Hourly. The lookback length is $L = 5H$.

Optimization. Adam with $\eta_0 = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, no weight decay. sMAPE loss is computed per-step on the 1024-window batch. The gradient budget is 10^5 total steps, with one Lightning epoch defined as 1,000 steps (≈100 paper-epochs).

Learning-rate schedule. MultiStepLR with 10 evenly spaced milestones at $\{10\text{k}, 20\text{k}, \ldots, 100\text{k}\}$ steps, $\gamma = 0.5$. The LR is therefore halved at each 10% boundary of the training budget. This is exactly the schedule used by Oreshkin et al.

Validation and early stopping. Validation is run every 100 training steps (`val_check_interval=100`) on the held-out window of each series. Early stopping monitors validation sMAPE with patience 15 (counted in val-check units, so a minimum of 1,500 steps must elapse before stopping can fire) and $\Delta_{\min} = 10^{-3}$. The sub-epoch validation cadence is the only departure from a strict line-by-line copy of Oreshkin et al. (2020); without it, early stopping fires on the first warmup-corrupted check and collapses to `best_epoch` $\in \{0, 1\}$.

Other settings. `n_blocks_per_stack = 1`, `share_weights = true`, ReLU activations throughout, `forecast_multiplier = 5`, default block widths (`g_width = 512`, `s_width = 2048`, `t_width = 256`).

A.2 Plateau LR variant (ReduceLROnPlateau)

The plateau variant (`comprehensive_m4_paper_sample_plateau.yaml`) is identical to A.1 in every respect except the learning-rate schedule. The motivation is that MultiStepLR commits to a fixed decay schedule independent of how training is actually progressing. A plateau scheduler can react to validation stalls and find a more aggressive effective decay on cells where the loss surface is well-behaved.

Learning-rate schedule. `ReduceLROnPlateau` with `factor = 0.5`, `patience = 3` (val-check units, so the LR is reduced after 3 consecutive 100-step val checks without improvement of at least $\Delta_{\min} = 10^{-3}$), `cooldown = 1` (one val-check skipped after each reduction before monitoring resumes), `min_lr =` 10^{-6}, `mode = min`, monitored on `val_loss`. The scheduler is stepped on the same cadence as validation (every 100 training steps) so that its trigger logic operates in the same time units as early stopping.

Effect. Empirically the plateau scheduler delivers more LR reductions than MultiStepLR on M4 Quarterly, Monthly, and Daily, and fewer on Yearly and Weekly, which matches the observed differences in the per-period leaderboards (Section 4).

All other settings (sampling, L_h, batch size, step budget, sub-epoch validation, early stopping, optimizer, loss, block widths, weight sharing, seed budget) are inherited unchanged from A.1.

A.3 Why two paper-sample protocols

The N-BEATS paper reports a single training procedure, so any one-protocol comparison conflates the architectural change with whatever artifacts that specific schedule introduces on the new architecture. By running each architecture under both A.1 (faithful) and A.2 (plateau) we can attribute observed gains to one of three categories. (i) Gains under both protocols are robust evidence for the architectural claim. (ii) Gains under the faithful protocol only constitute a head-to-head replacement for the published baseline. (iii) Gains under the plateau protocol only are a finding contingent on a more responsive LR schedule, which we report as such rather than as a clean architectural win.

B Hyperparameter grids

C Configuration list

D Additional ablations

E M4 full per-period tables

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.